ISG-NIS-Helper — Troubleshooting

1. First move, always

Double-click **`Run-Collect.bat`** on the affected machine.

It gathers one dated folder and one zip under `install-logs\`: machine and OS, recent Windows hotfixes, clock state, file inventory with hashes, both configs, the user list, the pre-scan report, both scheduled tasks in full, running processes, the ProgramData tree, `health.json` and `SAFEMODE.json`, recent sessions, the last few days of events, a fresh preflight run and all three self-tests.

`00-SUMMARY.md` is written to be pasted whole into a conversation.

PINs are redacted — the `pin` column and `master_pin` become `****` before anything is written. No image data is included; nothing is modified.

If the program folder is not writable (for example if you installed under `C:\Program Files` instead of the recommended `C:\ISG-NIS-Helper`), the bundle goes to `C:\ProgramData\ISG-NIS-Helper\install-logs` instead. The script prints where it wrote.

2. Symptoms

Symptom	Cause	Fix
Clicking the NIS-Elements icon does nothing for a moment, then opens the sign-in window	Normal: the launcher is waking the resident agent	No action; it opens within a few seconds
Sign-in window never appears when the icon is clicked	Agent task not running	Check `ISG-NIS-Helper-Agent` in Task Scheduler; look for `agent_started` in today's events. The launcher opens NIS anyway (fail-open) so the instrument is never stuck
The icon opens NIS-Elements with no sign-in	The gate failed open because no agent answered	Look for `launch_request_unhandled` then `nis_launched via launcher_failopen` in the events; fix why the agent task is not running
Sign-in works but NIS won't start: "requires elevation" (WinError 740)	NIS-Elements needs administrator rights	Expected on installs where NIS requires elevation — the helper launches it via ShellExecute, so Windows shows the normal UAC prompt; approve it. Same prompt you'd get double-clicking NIS directly
The NIS-Elements icon still opens NIS directly (no gate)	Install did not repoint that shortcut	Re-run `install.bat`; check the `NIS-Elements shortcuts` section of the install log for what matched
A configuration window appears instead	A `<<<SET_ME>>>` value remains	It names the key it is waiting on
NIS closes even though somebody signed in	The running executable is not in `nis_process_names`	Add the real name; restart `ISG-NIS-Helper-Svc`. Preflight checks for this
Sessions never end	Same cause, opposite symptom — ISG-NIS-Helper is not watching the process that is running	Same fix
Sessions end immediately	The watched name belongs to a launcher that exits	Check what is left in Task Manager once NIS is fully open
Sessions never end, *and* every launch is flagged	A permanently-running service is in `nis_process_names`	Remove it. `nissrv.exe` is Windows Defender
"No approved user list was found"	`users.csv` missing from the ISG-NIS-Helper folder	Create it from `users_template.csv`
A change to `users.csv` did not save	Program folder is under `C:\Program Files` (not the recommended `C:\ISG-NIS-Helper`)	Run Notepad itself as administrator, or reinstall under `C:\ISG-NIS-Helper`
Nothing in the log mirror	`network_root` unreachable	Local logs are complete; copies queue in `pending_sync` and go across later
A user forgot their PIN	—	Delete their file from `C:\ProgramData\ISG-NIS-Helper\pins\`
Install granted Users write over the executables	A copy of the program was sitting in `C:\ProgramData\ISG-NIS-Helper`	That folder is the log store; delete the stray copy and re-run install
`python -m build.bat` fails	It is a batch file, not a module	Type `build.bat`
Built agent dies at launch with a Tcl error	Conda's Tcl/Tk was not bundled	`conda install -n isg-nis-helper tk`, rebuild
Build fails with PermissionError on a `.spec`	Building inside a non-writable folder	Build in `C:\ISG-NIS-Helper-build`, install the output elsewhere

Somebody is waiting to image and everything looks broken. Open Task Scheduler, end `ISG-NIS-Helper-Agent` and `ISG-NIS-Helper-Svc`, and let them work. Both restart at the next logon or reboot, and the gap is visible in the log. There is deliberately no emergency off switch beyond this, because the program only reads and writes its own log files.

3. Failing safe

Windows updates, Defender definitions and driver changes arrive without warning, and you cannot enumerate in advance what they might disturb. So ISG-NIS-Helper degrades in one direction rather than anticipating specific faults.

The asymmetry it turns on. If the logger breaks you lose records — recoverable. If the enforcer breaks it may close NIS-Elements when it should not: overnight, mid-acquisition, unrecoverable. So the watcher's power to close a program is a privilege it has to keep earning. **Nothing below can stop ISG-NIS-Helper recording. It only governs whether ISG-NIS-Helper may close anything.**

Never act on a guess. A failed or implausible process enumeration returns *unknown*, not *empty*, and unknown means do nothing. (Empty used to mean "NIS closed" — so a transient fault could end a session and get a live acquisition killed.) Implausible matters too: a list of four entries containing none of the processes that exist on every running Windows machine is not a real answer.

Confirm before acting. A state must hold for `confirm_polls` consecutive readings — three, about nine seconds — before a session ends or a launch clock starts.

Preconditions on every closure, re-checked live: process list readable and plausible; state directory reads and writes; the NIS executable still on disk; the sign-in agent alive. Any failure and the closure does not happen, with `enforcement_withheld` and the reason in the log.

The agent check matters most. With a dead agent nobody can sign in, so enforcing would lock the instrument with its own logger. The agent proves it is alive by writing `agent.json` every few seconds — not by its executable name, so renaming things cannot silently disable this.

Windows update pause. The build number is tracked. On a change: log `os_build_changed`, run a self-check, and pause enforcement for 24 hours while recording continues. The hours after an update are exactly when odd behaviour appears. The pause expires by itself; `post_update_grace_hours = 0` disables it.

Safe mode. Three closures in a rolling hour, or ten internal errors in ten minutes, writes `SAFEMODE.json`, stops enforcement, keeps recording, and says so on the sign-in window. Three closures in an hour is far likelier to be ISG-NIS-Helper malfunctioning than three people abusing the instrument.

It clears only when a human deletes that file. This is not a manual kill switch: it cannot stop the recording, only the closing.

What this does not cover

An update that quarantines the executables or disables the scheduled tasks stops ISG-NIS-Helper entirely. That fails open — the scope still works — but silently, which is its real cost. Two mitigations:

- **Add the program folder to the Defender exclusion list.** PyInstaller output draws false positives; this is the most likely way an update stops it.
- **Read the log occasionally.** A month with no sessions on a busy scope means something stopped, and nothing else will tell you.

4. health.json

`C:\ProgramData\ISG-NIS-Helper\health.json` is rewritten every ten minutes: the last check results, safe-mode state, whether enforcement is paused, and closures this hour. It is the first file to read when somebody says ISG-NIS-Helper is behaving strangely.

5. Failsafe settings

Key	Default	Effect
`confirm_polls`	3	Agreeing readings before acting
`post_update_grace_hours`	24	Enforcement pause after a Windows update
`max_terminations_per_hour`	3	Closures before safe mode trips
`max_errors_per_10min`	10	Errors before safe mode trips
`require_agent`	true	Only enforce while the sign-in program is alive

6. Event kinds

Kind	Meaning
`agent_started`, `agent_stopped`	the sign-in program's lifecycle
`watch_started`, `watch_heartbeat`	the watcher is alive
`watch_self_check`	failsafe preconditions, at startup and every ten minutes
`watch_error`	the watcher hit an error and continued
`agent_not_heartbeating`	a session is open but the agent went quiet
`process_list_unreadable`	enumeration failed or looked implausible; nothing acted on
`user_list_loaded`	with `source`: `network`, `cache` or `none`
`signin_failed`	wrong PIN, or no PIN set
`pin_changed`	somebody set their own PIN
`session_start`, `session_end`	with the reason it ended
`session_resumed`	agent restarted while NIS was still open
`launcher_clicked`	the NIS-Elements shortcut was clicked
`launch_requested`	the resident agent picked up a shortcut click and opened sign-in
`launch_request_unhandled`	no agent answered in time; the launcher opened NIS itself (fail-open)
`agent_already_running`	a second resident agent exited so only one runs
`nis_launched`, `nis_closed`	the program starting and stopping
`nis_launch_failed`	the configured path is wrong
`unauthorized_launch_detected`	NIS opened with nobody signed in
`unauthorized_launch_closed`	and it was closed after the grace period
`enforcement_withheld`	a closure was due but a precondition failed
`os_build_changed`	Windows updated; enforcement paused, logging continued
`post_update_self_check`	the checks run immediately after an OS update
`safe_mode_entered`, `safe_mode_cleared`	a circuit breaker tripped, or somebody cleared it
`baseline_snapshot`	the start-of-session file index completed
`diff_failed`, `diff_skipped`	file changes could not be recorded
`sync_flushed`	queued copies reached the share
`training_request_clicked`	somebody used the contact button